

Page fault

In computing, a **page fault** is an exception that the memory management unit (MMU) raises when a process accesses a memory page without proper preparations. Accessing the page requires a mapping to be added to the process's virtual address space. Furthermore, the actual page contents may need to be loaded from a back-up, e.g. a disk. The MMU detects the page fault, but the operating system's kernel handles the exception by making the required page accessible in the physical memory or denying an illegal memory access.

Valid page faults are common and necessary to increase the amount of memory available to programs in any operating system that uses virtual memory, such as Windows and macOS.^[1]

Types

Minor page fault

If the page is loaded in memory at the time the fault is generated, but is not marked in the memory management unit as being loaded in memory, then it is called a *minor*, or *soft*, page fault. The page fault handler in the operating system merely needs to make the entry for that page in the memory management unit point to the page in memory and indicate that the page is loaded in memory; it does not need to read the page into memory. This could happen if the memory is shared by different programs and the page has already been brought into memory for another program.

Major page fault

This is the mechanism used by an operating system to increase the amount of program memory available on demand. The operating system delays loading parts of the program from disk until the program attempts to use them, at which time page faults are generated. If the page is not loaded in memory at the time of the fault, then the fault is *major*, or *hard*. The page fault handler in the OS needs to load the page into working memory, and to do so it must choose a location: either a free page or, when free space is scarce, a page in use by another process. In the latter case the OS needs to write the data in that page out to mass storage (if it has not been written out since it was last modified) and mark that page as not being in memory in its process page table. Once the space has thus been made available, the OS reads the data for the new page into memory, adds an entry for its location in the memory management unit, and indicates that the page is loaded. Thus, major faults are more expensive than minor faults and add storage access latency to the interrupted program's execution.

Invalid page fault

If a page fault occurs for a reference to an address that is not part of the virtual address space, meaning there cannot be a page in memory corresponding to it, then it is called an *invalid* page fault. The page fault handler in the operating system will then generally pass a segmentation fault to the offending

process, indicating that the access was invalid; this usually results in abnormal termination of the code that made the invalid reference. A null pointer is usually represented as a pointer to address 0 in the address space; many operating systems set up the MMU to indicate that the page that contains that address is not in memory, and do not include that page in the virtual address space, so that attempts to read or write the memory referenced by a null pointer get an invalid page fault.

Invalid conditions

Illegal accesses and invalid page faults can result in a segmentation fault or bus error, resulting in an app or OS crash. Software bugs are often the causes of these problems, but hardware memory errors, such as those caused by overclocking, may corrupt pointers and cause valid code to fail.

Operating systems provide differing mechanisms for reporting page fault errors. Microsoft Windows uses structured exception handling to report invalid page faults as access violation exceptions. UNIX-like systems typically use signals, such as SIGSEGV, to report these error conditions to programs. If the program receiving the error does not handle it, the operating system performs a default action, typically involving the termination of the running process that caused the error condition, and notifying the user that the program has malfunctioned. Windows often reports such crashes without going to any details. An experienced user can retrieve detailed information using WinDbg and the minidump that Windows creates during the crash. UNIX-like operating systems report these conditions with such error messages as "segmentation violation" or "bus error", and may produce a core dump.

Performance impact

Page faults degrade system performance and can cause thrashing. Major page faults on a conventional computer using hard disk drives can have a significant impact on performance, as a typical hard disk drive had an average rotational latency of 3 ms, a seek time of 5 ms and a transfer time of 0.05 ms/page. Therefore, the total time for paging is near 8 ms (8,000 μ s). If the memory access time is 0.2 μ s, then the page fault would make the operation take about 40,000 times as long. With a more modern system using a fast solid-state drive with a page read latency of 0.030 ms (30 μ s)^[2] and a memory access latency of 70 ns (0.070 μ s),^[3] a hard page fault is still takes over 400 times as long.

Performance optimization of programs or operating systems often involves efforts to reduce the number of page faults, by reducing overall memory usage or by improving memory locality. To minimize page faults, OS developers use an appropriate page replacement algorithm that maximizes page hits. Many have been proposed, such as implementing heuristic algorithms to reduce the incidence of page faults.

Having more physical memory also reduces the likelihood of page faults. Chunks of memory-mapped files can remain in memory longer and avoid slow re-reads from storage. Similarly, having more memory lessens the need for frequent swapping out of memory pages to a backing storage device.

See also

- Bélády's anomaly

- Memory hierarchy

References

1. Bovet, Daniel; Cesati, Marco (November 2005). *Understanding the Linux Kernel* (<https://web.archive.org/web/20231224220514/http://gauss.eecs.uc.edu/Courses/c4029/code/memory/understanding.pdf>) (PDF) (3rd ed.). O'Reilly Media. ISBN 0-596-00565-2. Archived from the original (<http://gauss.eecs.uc.edu/Courses/c4029/code/memory/understanding.pdf>) (PDF) on 24 December 2023. Retrieved 9 October 2021.
2. W1zzard (2023-08-08). "Samsung 990 Pro 2 TB Review" (<https://www.techpowerup.com/review/samsung-990-pro-2-tb/5.html>). *TechPowerUp*. Retrieved 2025-05-19.
3. W1zzard (2024-08-07). "AMD Ryzen 5 9600X Review - The Best Sub-\$300 Gaming CPU" (<https://www.techpowerup.com/review/amd-ryzen-5-9600x/6.html>). *TechPowerUp*. Retrieved 2025-05-19.

Bibliography

- John L. Hennessy, David A. Patterson, *Computer Architecture, A Quantitative Approach* (ISBN 1-55860-724-2)
- Tanenbaum, Andrew S. *Operating Systems: Design and Implementation (Second Edition)*. New Jersey: Prentice-Hall 1997.
- *Intel Architecture Software Developer's Manual*—Volume 3: System Programming

External links

- "So What Is A Page Fault? (<https://web.archive.org/web/20061016080158/http://www.osronline.com/article.cfm?article=222>)" (subscription required) from OSR Online (a Windows-specific explanation)
- "Virtual Memory Details (https://access.redhat.com/documentation/en-US/Red_Hat_Enterprise_Linux/4/html/Introduction_To_System_Administration/s1-memory-virt-details.html)" from the Red Hat website.
- "UnhandledExceptionFilter (Windows) (<http://msdn2.microsoft.com/en-us/library/ms681401.aspx>)" from MSDN Online (<http://msdn.microsoft.com/library>).
- "Page fault overhead (<https://plus.google.com/+LinusTorvalds/posts/YDKRFDwHwr6>)" for information about how page faults can crucially affect processing time.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Page_fault&oldid=1356205468"